Towards Light Weight Semantic Segmentation for Mars Navigation

Gene Foxwell

Woolf University / Udacity School of Artificial Intelligence

Deep Learning Systems

Overview

This paper looks at different approaches for creating a semantic segmentation model that a Mars Rover, such as Curiosity, could use for navigational purposes. We start by building a standard UNet model as a baseline. Then we look at different methods of reducing the model size and the effects of feature engineering on the results. Finally, we compare the results, discuss ethical concerns and biases, before closing off with a note about possible future work.

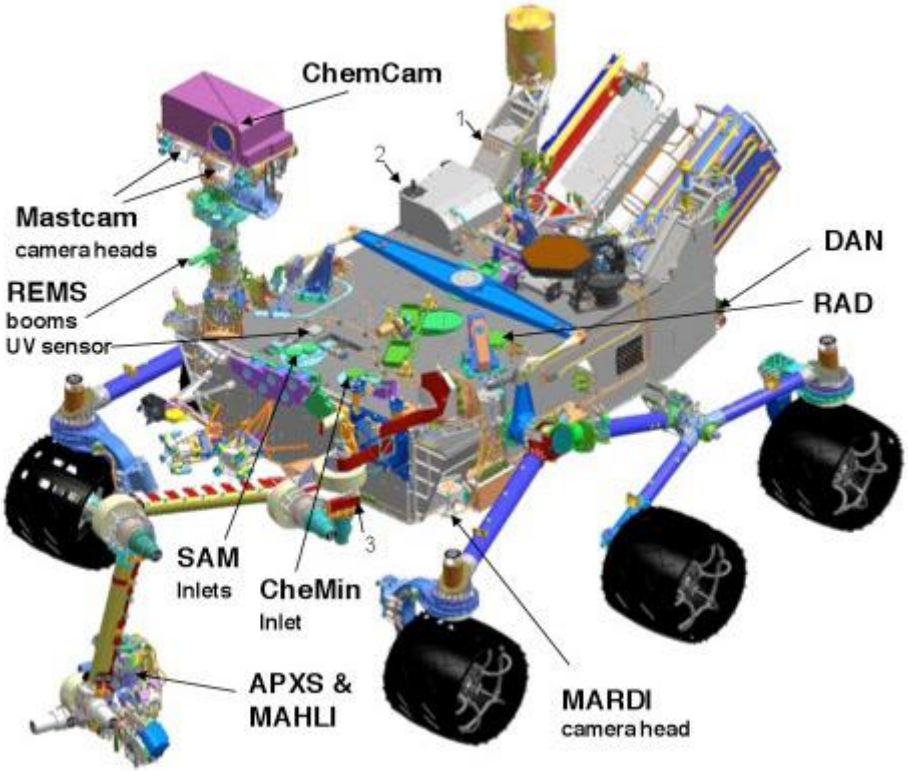


Figure 1 NASA/JPL-CalTech

Dataset and Task Description

Our task was to build an efficient semantic segmentation model that determines which pixels in the image are safe for the rover to transverse, and which ones are unsafe. Our approach to building a more efficient model will be to attempt to reduce its size. Data collected (Yang, C

2023) suggests that smaller models drain less power than larger models. They also required less storage, and fewer FLOPS to run. These are important considerations for use in an autonomous environment on another planet.

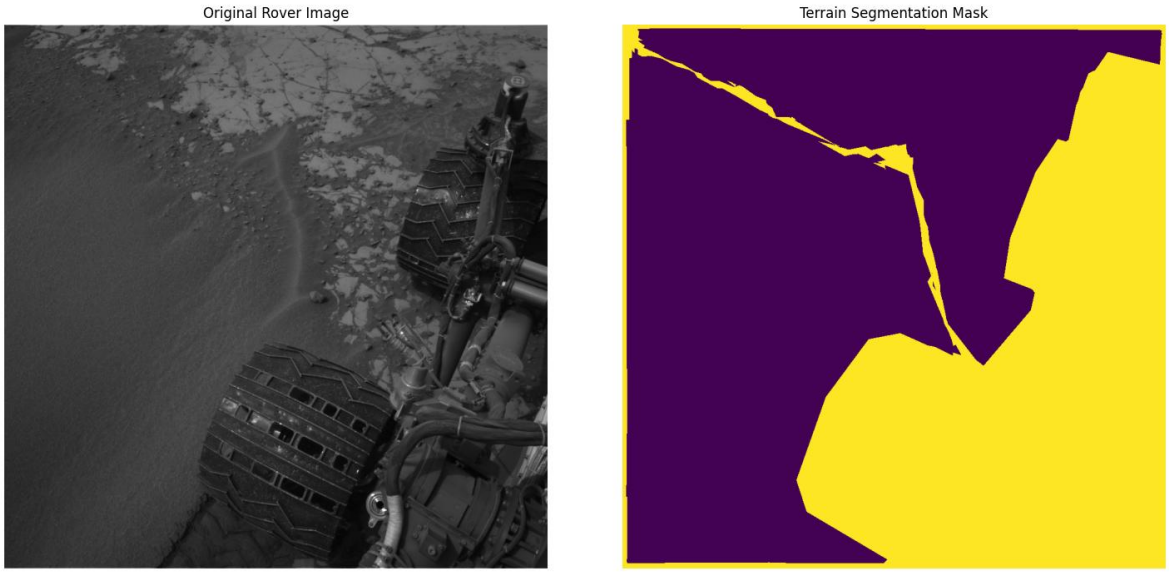
To solve this problem, we will use the AI4Mars dataset (https://www.kaggle.com/datasets/darshannnnnn/ai4mars-raw). We focus on the subset of the data concerning the images taken from the cameras on Curiosity Rover's mast. Isolating this data provides us with 16064 images and labels.

Labels for the AI4Mars dataset were created via crowd sourcing. Users were shown a tutorial on how to identify various features of the Martian landscape and then asked to label the images. Further information can be found in (Swan, R 2021).

AI4Mars labels were in four categories: 0: soil; 1: bedrock; 2: sand; 3: big rock; 255: null

To convert this into a binary classification problem, we remapped the labels onto:

- 0,1 → Transversable (1)
- 2,3,255 → Non-Transversible (0).



Model Architecture and Design Decisions

The primary model we are starting with is the UNet (Ronneberger 2015). UNet consists of two main components – an Encoder and a Decoder – linked by convolution layer. Each layer of the decoder has a skip connection to a corresponding layer on the decoder. See diagram below for an overview of the UNet architecture. This model has 15,329,473 parameters.

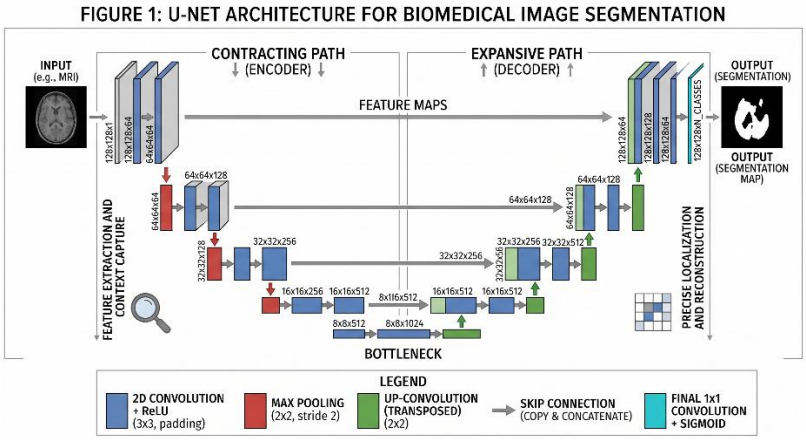


Figure 2 UNet (Generated with Assistance of Gemini)

Our chosen task was to build a light-weight model for Rover Navigation. To this end, we constructed two additional variations of the UNet architecture:

- **Mini-UNet:** This model was constructed by reducing the number of filters in each layer of the UNet by half. The resulting model had 960,049 parameters.
- **Compressed Mini-UNet:** This model was constructed by reducing the number of filters in each layer of the UNet to one quarter of the original. The resulting model had 240,655 parameters.

We started by training a standard UNet to serve as our baseline. As it's our baseline network; the only transformations we made were to ensure the image was greyscale and reduce its size to 512x512. When transforming the masks we used a Nearest-Neighbor scaling method to avoid corrupting the labels.

We split the data into three sets: 70% of the dataset was used for training, 15% for validation, and 15% for evaluating the results.

Dataset	**Size**
Train	11244
Validation	2409
Test	2411

Training was done using the Adam Optimizer, a Learning Rate of 3e-4, batch size of 8, and five epoch using a NVIDIA GeForce 5070 Graphics Card. The same hyper parameters and optimizer were used for all models.

Each training session used the same loss function – Dice Loss (Jadon, S 2020). This loss function measures the similarity between two images. In this case we use it to measure how similar the model's output image is compared to the ground truth mask.

$$DL(y,p) = 1 - \frac{(2yp + 1)}{y + p + 1}$$

Here "y" is the ground truth and "p" is the predicted image from the model.

We further evaluated the results using the mean Intersection Over Union Metric (mIoU) (Long, J 2015)

$$IoU_k = \frac{TP_k}{TP_k + FP_k + FN_k}$$

$$mIoU = \frac{1}{N}\sum_k IoU_k$$

This takes the number of correct pixels from each predicted image and compares that to the ratio of true positive pixels + false positive pixels + false negative pixels. This is done for each image in the training set. The final score is the mean of the individual results in the set. This gives a fine-tuned measure of the model's performance – evaluating its results on a pixel by pixel basis.

Finally, we plotted five sample images side-by-side with their original training masks and the predictions of the model.

Experimental Comparison

The standard UNet was training with minimal transformations to the input data – no data augmentation was performed. Specifically, we:

- Transformed the image to gray scale.
- Resized the images to 512x512

This gave us a baseline to compare our changes.

It was hypothesized that we could compensate for the smaller model sizes by preprocessing the images using OpenCV. The use of traditional approaches combined with Deep Learning approaches was motivated by Neurosymbolic AI (Colelough, B 2025). In this case, we wished to use OpenCV to extract boundaries between objects using Canny Lines. To do this we have created a new transformation pipeline:

- OpenCV Gaussian Blur to clean up image noise.
- Canny Line to extract the edges from the image.
- Morphological Dilation to make the edges more apparent.

These filters were applied to the data in three different ways (creating four datasets per model)

- **Standard Dataset:** No OpenCV transformations were applied.

- **Dual-Channel Dataset:** The results of the OpenCV transforms were given to the model on a separate channel.
- **OpenCV Only:** Only the transformed results of the images were shown to the model.
- **OpenCV + Standard Dataset:** Results of the OpenCV transform were superimposed on the original images.

Results and Interpretation

We measured the results of each Experiment by calculating the mIoU and Dice Score on the test set. We have output five examples of the models output from the test set for each variation. As these images are quite large, we've attached them to the end of the paper in the appendix.

Standard UNet

Data	Training Time	Testing Time	mIoU	Dice Score
Original data	528m 34s	1m 30s	0.76	0.86

Mini-UNet

Data	Training Time	Testing Time	mIoU	Dice Score
Original Data	31m 15.9s	1m 3.9s	0.76	0.86
Dual Channel	34m 24.4s	1m 11s	0.75	0.85
OpenCV Only	32m 8.6s	1m 6.7s	0.69	0.82
Original with OpenCV	32m 51s	1m 7.9s	0.73	0.84

Compressed Mini-UNet

Data	Training Time	Testing Time	mIoU	Dice Score
Original Data	30m 21s	1m 2.8s	0.76	0.86

Dual Channel	33m 45.8s	1m 6.6s	0.73	0.84
OpenCV Only	30m 41s	1m 4.7s	0.69	0.82
Original with OpenCV	31m 6.9s	1m 4.7s	0.72	0.84

Similar approaches to this problem (Dai, Y 2022) have achieved an mIoU between ~0.7 and ~0.8, which is consistent with the scores we have achieved for all our models.

Both the Mini-UNet and the Compressed Mini-UNet were able to match the original networks mIoU and Dice Scores using the unaltered dataset. This suggests that the full UNet is more powerful than needed for the problem at hand.

Training times for the smaller networks were significantly lower than that required for the full UNet (~8.8 hours for the full-sized network down to ~30 minutes). Inference time over the test set was also reduced by about 30 seconds overall, suggesting that the smaller networks also performed faster inference. This smaller network is trained significantly faster, does faster inference, and still scores as high as the original model on Dice and mIoU. This was a surprising result, as some degradation in the performance on smaller models was expected.

It was hoped that the OpenCV data would help boost the smaller networks' performance to make up for their reduced size. As these networks performed well without the augmented data, it seems that the OpenCV data may have hindered the system rather than helped. However, looking at the segmentation results, specifically the two large rocks in the last row of each example, it appears that only networks augmented with OpenCV data had detected these. It's possible that a tradeoff occurred between accurately detecting rocks of this kind and some other feature in the data.

Based on our metrics, the best model we explored for this task was Compressed Mini-UNet with the original dataset. Not only was the Compressed Mini-UNet smaller and faster than

its competitors, but training on the original dataset obtained similar metrics as were obtained on the full-sized network.

That said, although there is a drop in performance when using the OpenCV transforms, these networks do appear to find features in the data that the others miss – as the image with the two large rocks shows. For navigational purposes, we may be better off choosing a lower performing model if it is better at avoiding particularly dangerous obstacles that the higher performing models miss.

Limitations and Risks

We limited ourselves to data collected by Curiosity Rover NCAM data. While this already gave us 16,064 source images and labels, it was only a small portion of the available data. Furthermore, this data was limited only to that portion of the Martian Surface that the Curiosity Rover experienced, which accounts for a small portion of the planet's surface overall.

While our experimentation looked at augmenting the data with OpenCV based filters, as well as gradually reducing network size – we kept the OpenCV transformations constant when used. We did not explore additional settings on the filters, nor did we explore the possibility of adding new filters / removing existing filters from the transformation workflow. It's possible that by fine-tuning these parameters the models training using OpenCV enhanced data may have surpassed their non-enhanced cousins.

We were also limited by the hardware available. While the NVIDIA GeForce 5070 is reasonable for home use, its primary intended use is as a gaming GPU, not as dedicated hardware

for this purpose. With dedicated hardware, a larger dataset could've been trained in the time available.

As these models were trained using only Curiosity data, they should not be relayed upon as is for navigational purposes. We already see at least one case where a model has missed two giant rocks which if hit could seriously damage a Rover's delicate equipment. Instead, these should be used as a starting place for designing low-power models to enhance a future Rover's autonomous navigation abilities.

Ethical and Responsible Use

As hinted at above – these models are not intended to be used directly on an autonomous Rover – or any other autonomous vehicle. They have been trained on potentially limited datasets and can still miss important environmental hazards.

These models are heavily biased towards recognizing features found during the Curiosity Rovers Mars Mission. Terrain types and dangers that may have been more prevalent on other parts of Mars or even in other Mars missions may not be suitably accounted for here.

The ethical concern with misusing these models could be the potential loss of an exploration vehicle. Given that these are one-of-a-kind science platforms that provide data on the Martian landscape to researchers all over the world, it behooves us to ensure that any autonomous model used in them is trained and tested as rigorously as possible.

Future Improvements

To continue this work forward, the next practical step would be to start varying the OpenCV filter transformations. Only one stack of transformations was tried here, and its possible

given how close the metrics where for dual channel and combined channel approaches that with the right parameter choices these models could exceed the base model in performance.

One could also further reduce the size of the UNet, but based on the data collected so far, we suspect this would lead to increasingly diminishing returns.

Additional time, or access to dedicated hardware would allow this research to be expanded to a larger subset of the AI4Mars data. The AI4Mars data has data from three separate Mars missions across four different probes/Rovers – by combining this data we may be able to improve the results significantly.

Further experimentation with the number of epochs, learning rate, and optimizer would also be prudent as these were held constant across the models tested in this paper.

References

Dai, Y., Zheng, T., Xue, C., & Zhou, L. (2022). SegMarsViT: Lightweight Mars Terrain Segmentation Network for Autonomous Driving in Planetary Exploration. *Remote Sensing*, *14*(24), 6297. https://doi.org/10.3390/rs14246297

Jadon, S. (2020). A survey of loss functions for semantic segmentation. *2020 IEEE International Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*, 1–7. https://doi.org/10.1109/CIBCB48159.2020.9277638

Long, J., Shelhamer, E., & Darrell, T. (2015). Fully convolutional networks for semantic segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3431–3440. https://doi.org/10.1109/CVPR.2015.7298965

Colelough, B. C., & Regli, W. (2025). *Neuro-symbolic AI in 2024: A systematic review*. arXiv. https://doi.org/10.48550/arXiv.2501.05435

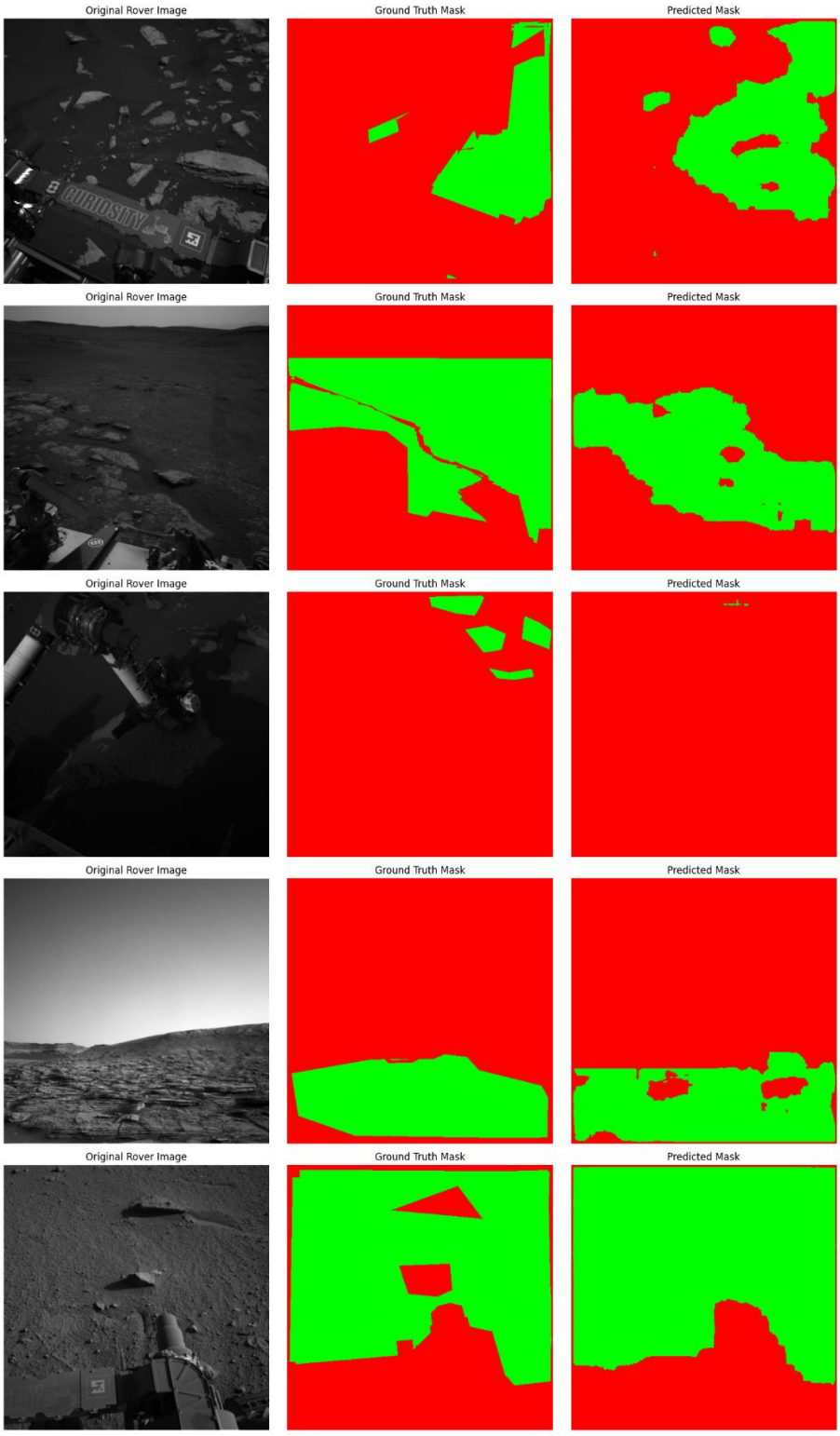
Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. *arXiv*. https://arxiv.org/abs/1505.04597

Swan, R. M., Kiri, L., Ono, M., Rothrock, B., & Higa, M. (2021). AI4MARS: A dataset for terrain-aware autonomous driving on Mars. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW 2021)*. https://openaccess.thecvf.com/content/CVPR2021W/AI4Space/html/Swan_AI4MARS_A_Dataset_for_Terrain-Aware_Autonomous_Driving_on_Mars_CVPRW_2021_paper.html

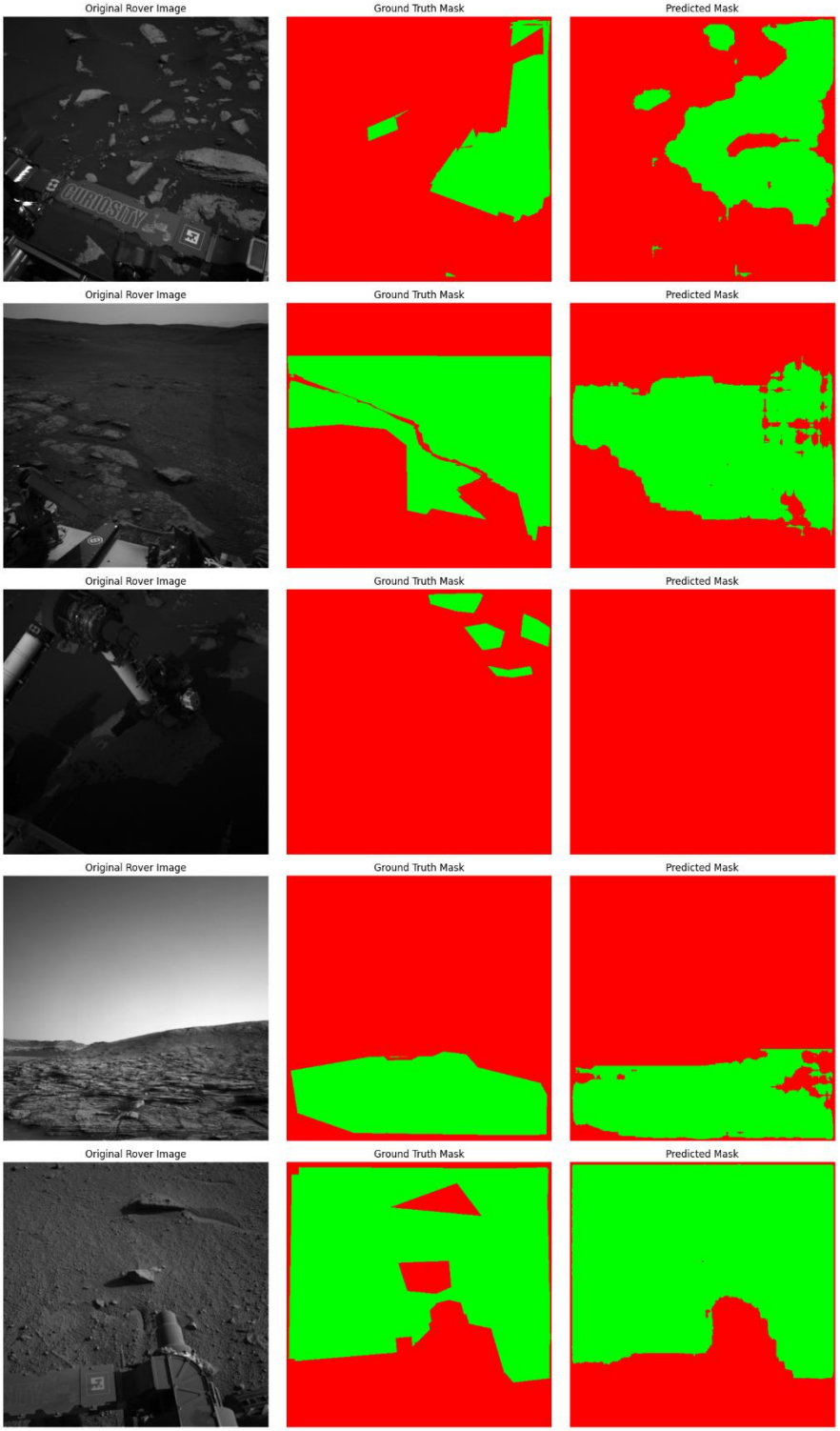
Yang, C., Wen, J., & Liu, Y. (2023). Unveiling energy efficiency in deep learning: Measurement, prediction, and scoring across edge devices. *arXiv*. https://arxiv.org/pdf/2310.18329

Appendix: Segmentation Images

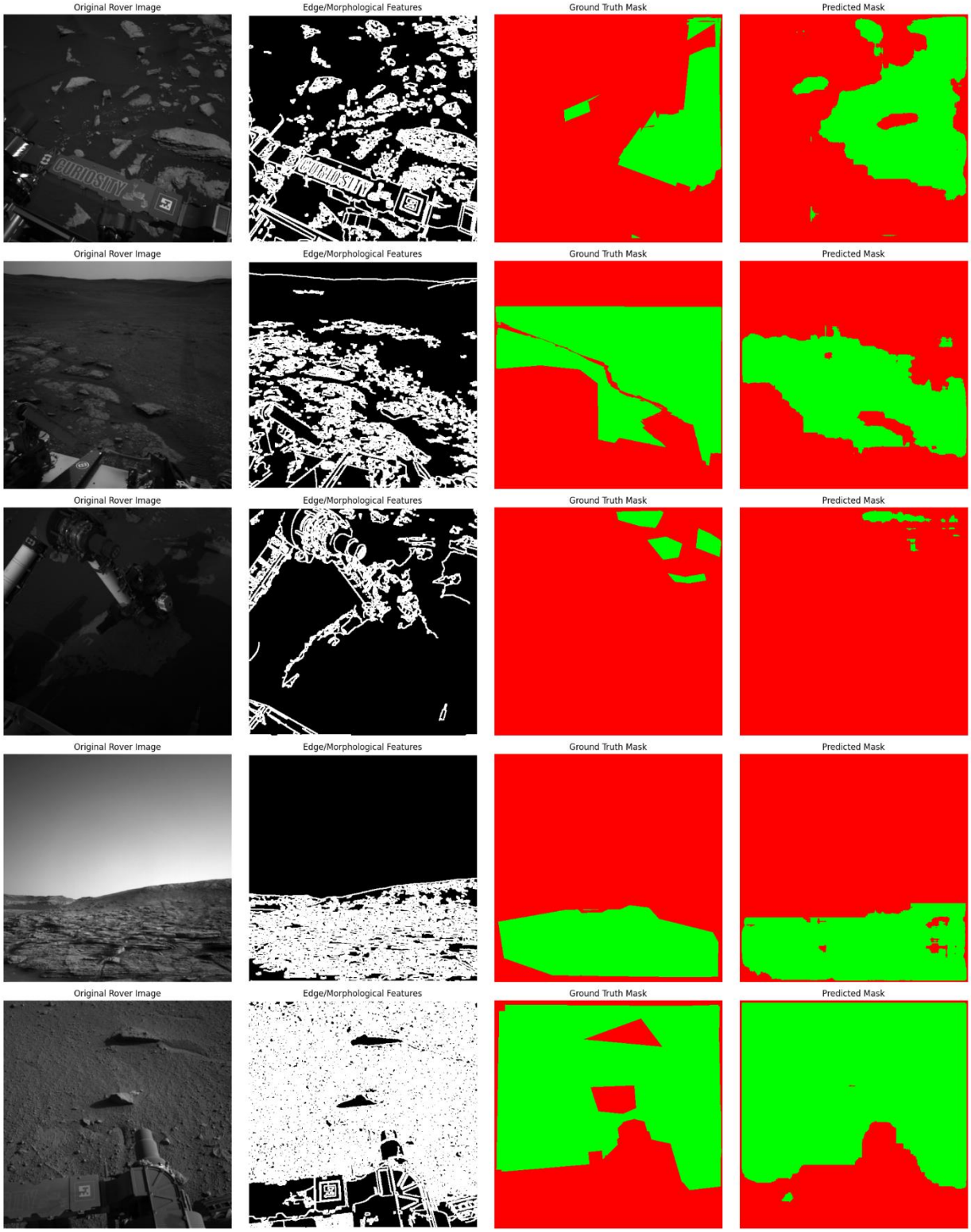
Standard UNet



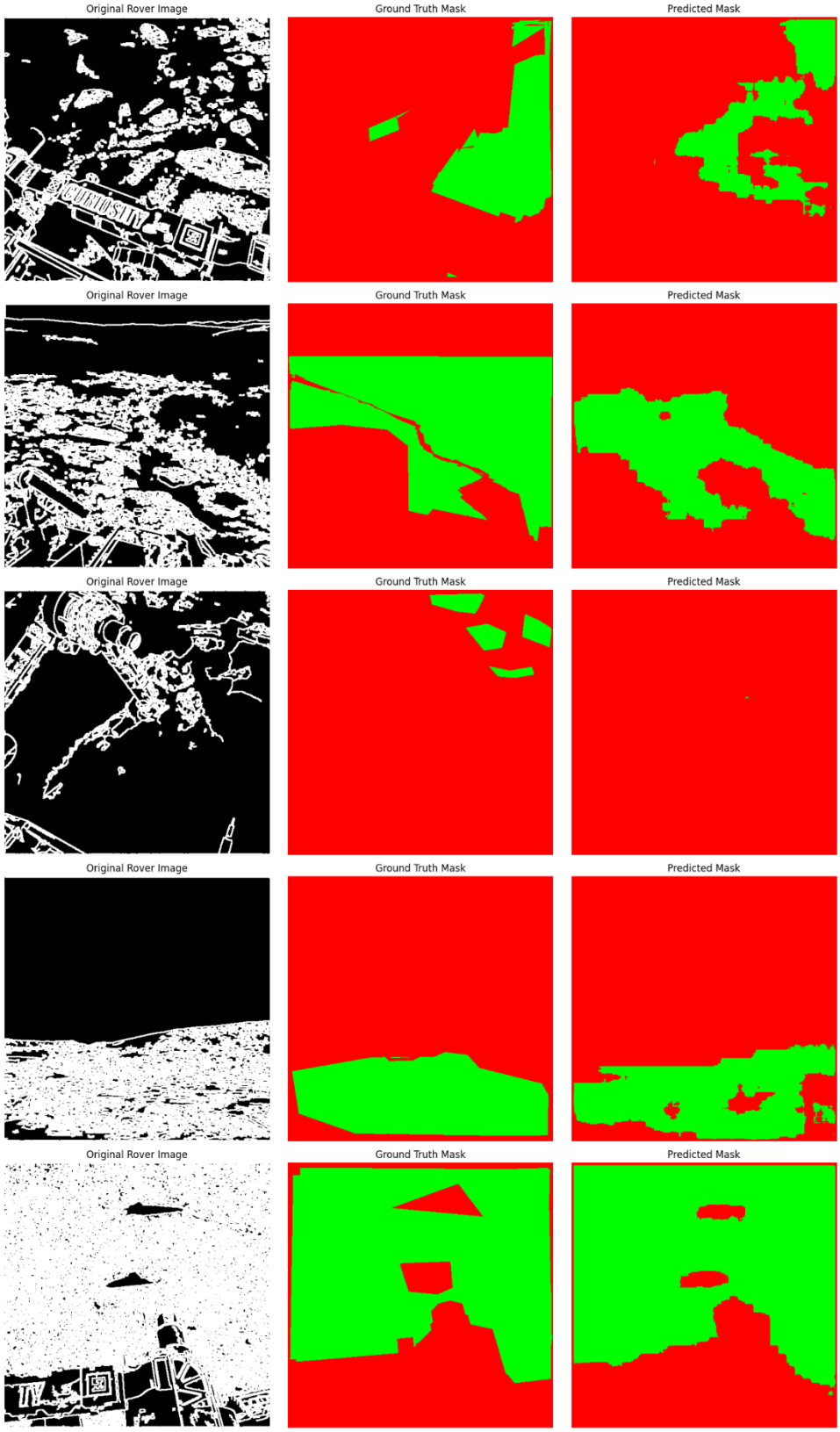
Mini UNet with Standard Data



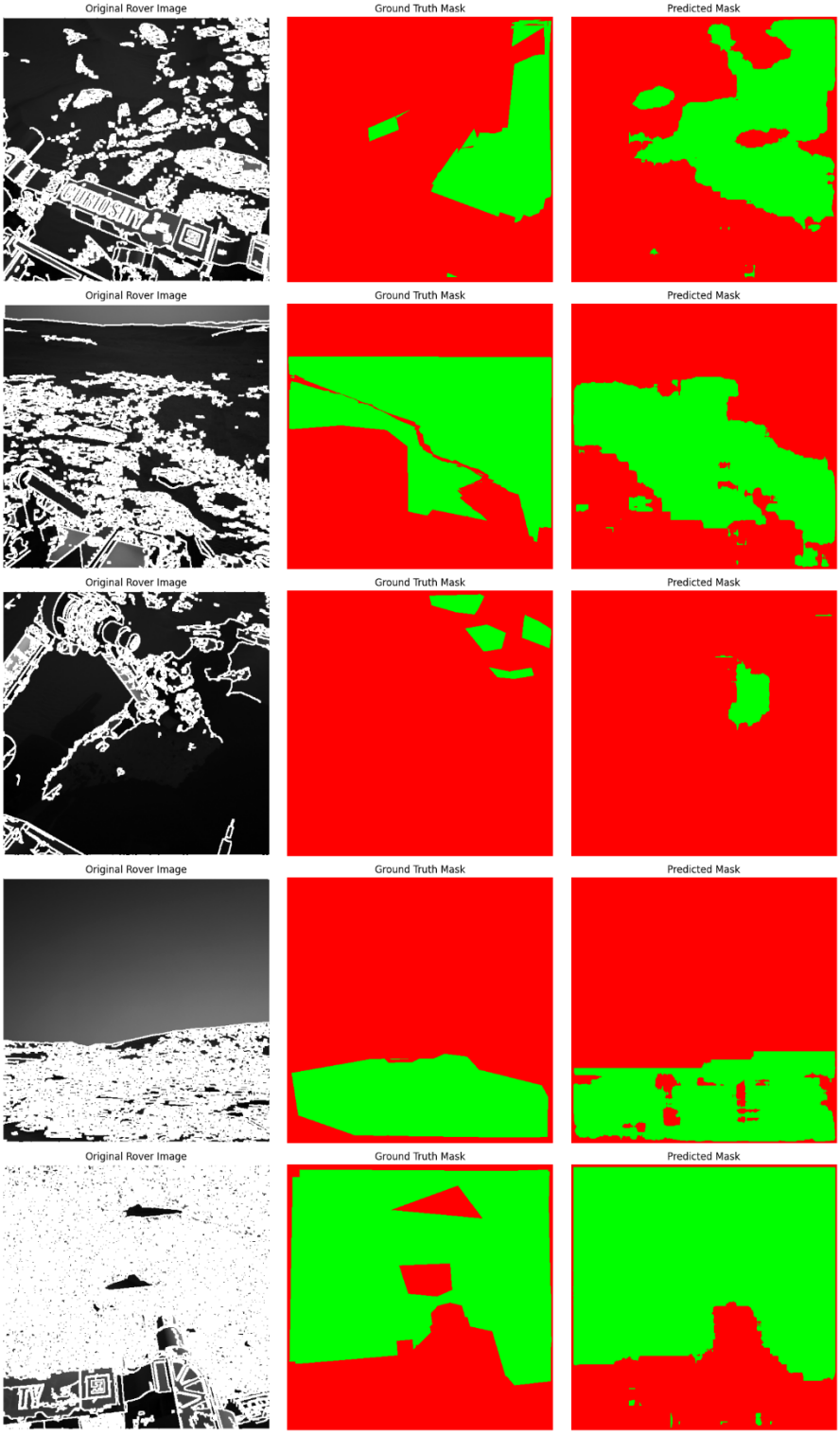
Mini UNet with Two Channels of Data



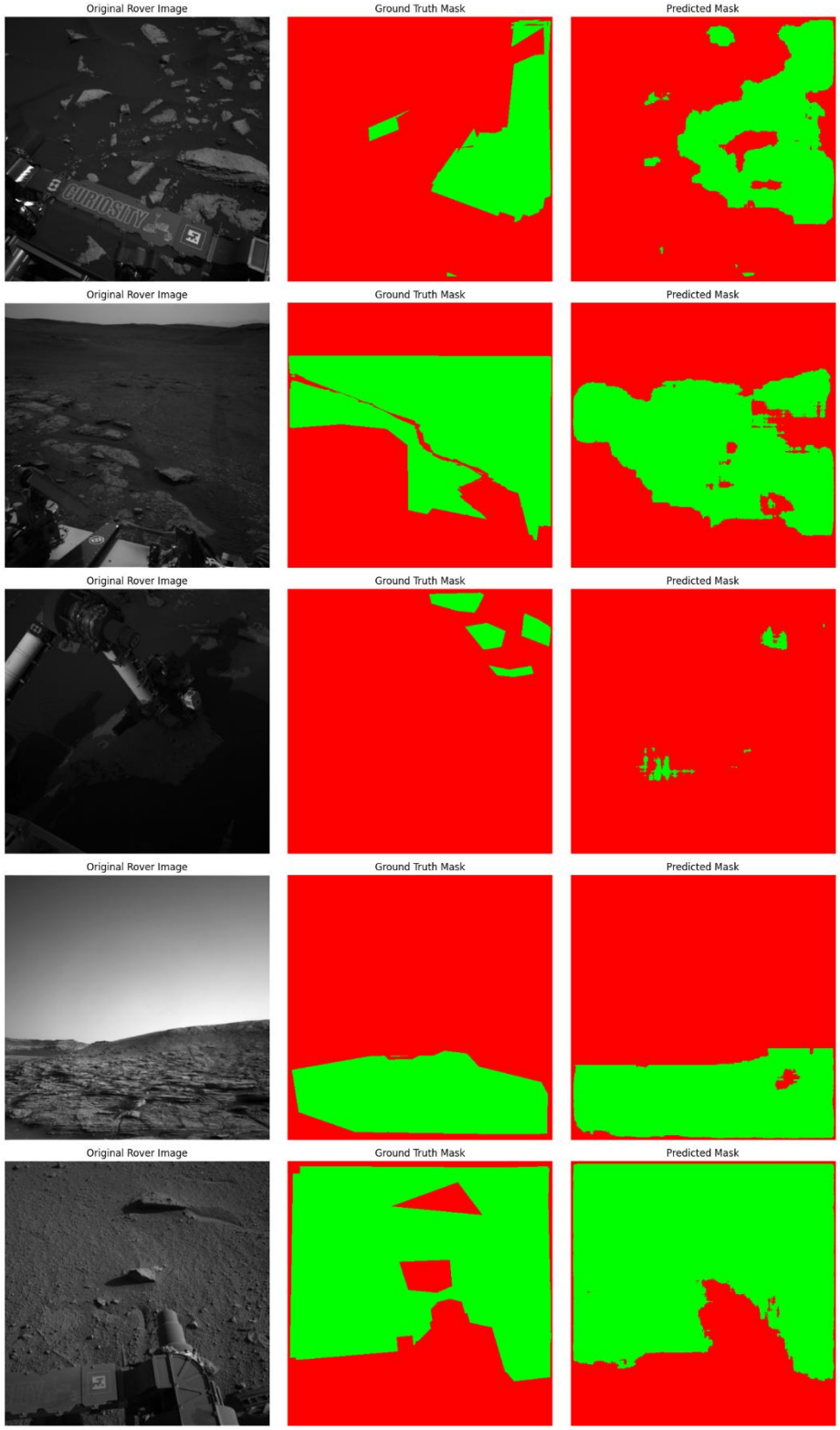
Mini UNet with OpenCV Transform Channel Only



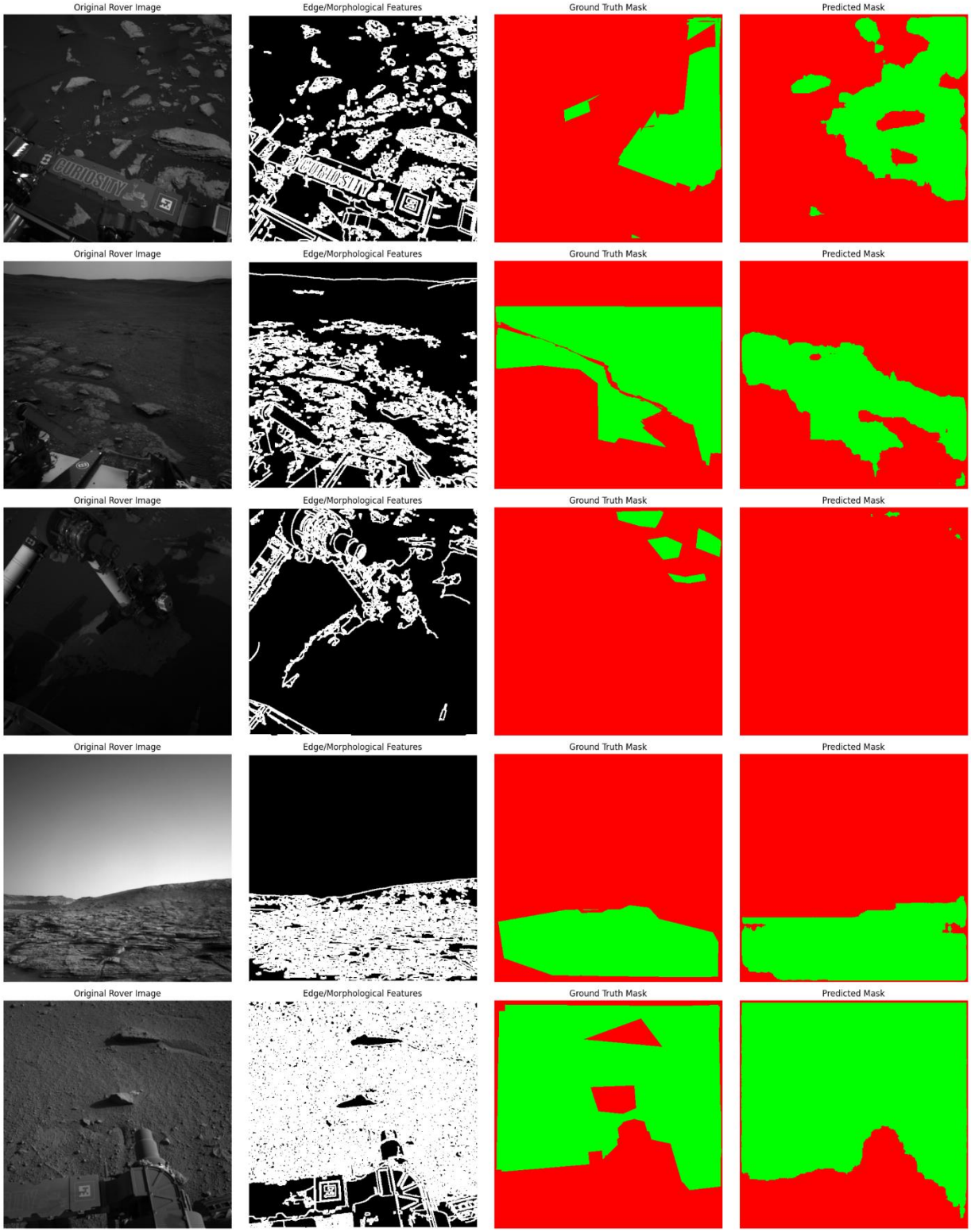
Mini UNet with Open CV and Standard Data Merged



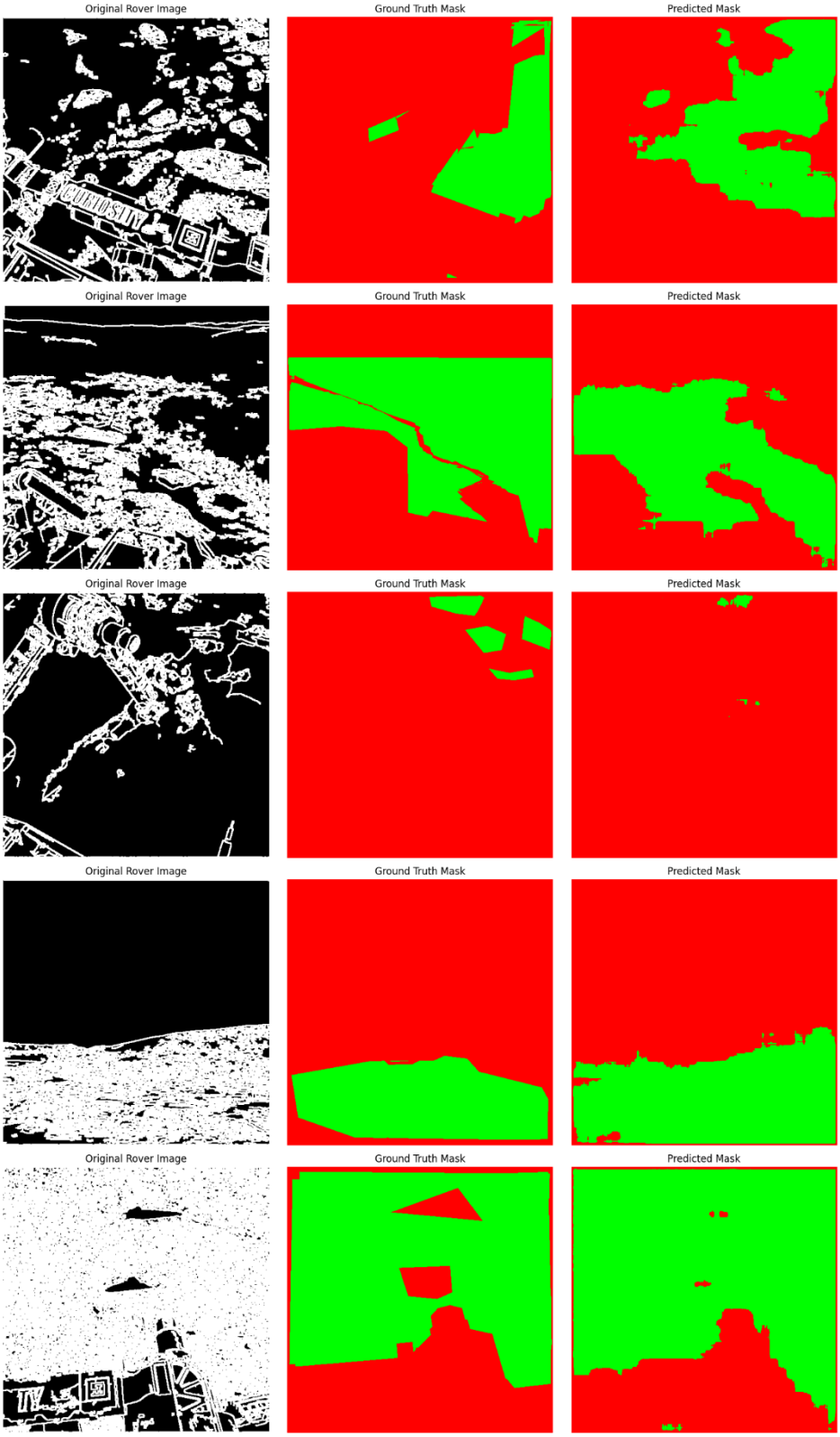
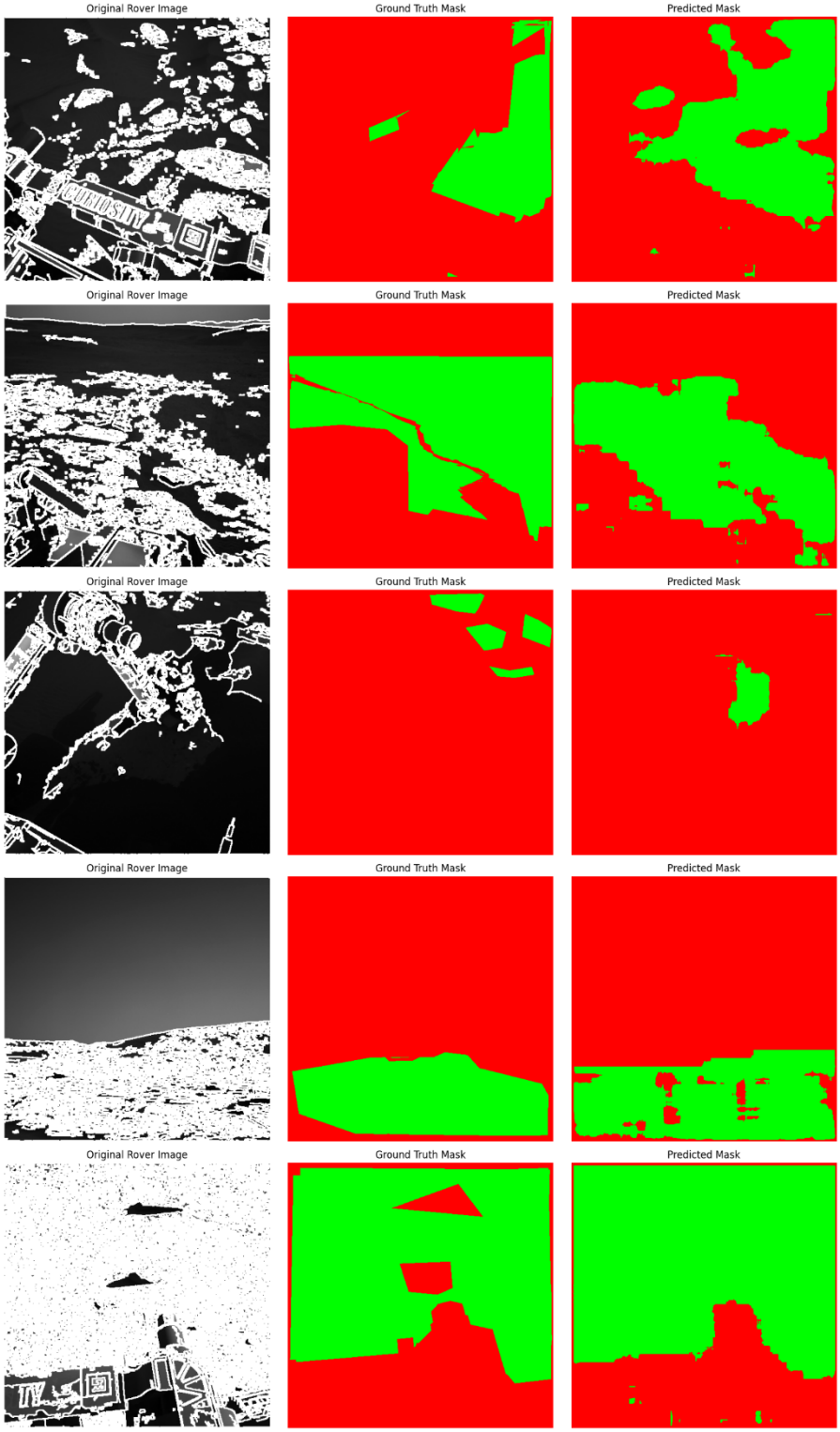
Compressed UNet with Original Data



Compressed UNet with Two Channels of Data



Compressed UNet with OpenCV Transform Channel Only



Compressed UNet with Open CV and Standard Data Merged